Session 2 — Controllers In Depth

Sessions 1 and 2 opened the box and named the four seams. This session lives entirely inside **seam 3** — the handler — and stays inside the one shape you'll actually write day to day: `@Controller`.

0. **What makes `@Controller` work at all** — the zero-config defaults, `<mvc:annotation-driven/>`, `@EnableWebMvc`
1. Request mapping — `@RequestMapping` and its shortcuts, path patterns, query params
2. Getting data **in** — `@RequestParam`, `@PathVariable`, `@RequestBody`
3. Command objects and data binding — `WebDataBinder`, `@InitBinder`
4. Forms with JSP tags — the Spring form tag library
5. Validation — `@Valid`, `BindingResult`, custom constraints
6. Exception handling — `@ExceptionHandler`, `@ControllerAdvice`
7. `HandlerInterceptor` **in depth** — where it sits, and why it exists next to Filters

---

0 · What Makes `@Controller` Work At All

The Question Nobody Asks Until It Breaks

You write `@Controller` and `@GetMapping`. Something has to read those annotations and something has to invoke that method. Session 1 named them: `RequestMappingHandlerMapping` and `RequestMappingHandlerAdapter`.

So do you have to declare them?

No. And the reason matters, because it decides what you can and cannot configure.

Level 0 — `DispatcherServlet.properties`

Inside `spring-webmvc.jar`, next to `DispatcherServlet.class`, sits a properties file listing a fallback for every strategy interface. In Spring 6 it reads:

Strategy	Defaults, in order
`HandlerMapping`	`BeanNameUrlHandlerMapping`, **`RequestMappingHandlerMapping`**, `RouterFunctionMapping`
`HandlerAdapter`	`HttpRequestHandlerAdapter`, `SimpleControllerHandlerAdapter`, **`RequestMappingHandlerAdapter`**, `HandlerFunctionAdapter`
`HandlerExceptionResolver`	`ExceptionHandlerExceptionResolver`, `ResponseStatusExceptionResolver`, `DefaultHandlerExceptionResolver`
`ViewResolver`	`InternalResourceViewResolver`
`LocaleResolver`	`AcceptHeaderLocaleResolver`
`RequestToViewNameTranslator`	`DefaultRequestToViewNameTranslator`
`FlashMapManager`	`SessionFlashMapManager`
`MultipartResolver`	**— none. No default at all.**

So `@Controller` works against a completely empty XML file. The mapping and the adapter are both in that list.

Two consequences worth holding on to:

- **Multipart is the exception.** It is the one strategy with no fallback, which is why Section 2's upload subsection needs an explicit bean and nothing else in this course does.
- **The list is consulted per type, and it is all-or-nothing.** Declare a single `HandlerAdapter` bean and the *entire* adapter default list is skipped — not merged with yours. That is why the four-components demo has to declare four adapters by hand, and it is also why dropping `<mvc:annotation-driven/>` into an old project can kill working `Controller`-interface beans overnight.

What Level 0 Does *Not* Give You

The defaults are instantiated by `DispatcherServlet` itself, not registered in your context. They work, but you cannot reach them. No message-converter customization, no extended `ConversionService`, no `@Valid` wiring, no interceptor registry, no content-negotiation configuration, no static-resource handling.

That gap is exactly what the next two levels exist to close.

Level 1 — `<mvc:annotation-driven/>`

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/mvc
           https://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <mvc:annotation-driven/>

</beans>
```

The namespace declaration and the schema location are half the work — a missing `xmlns:mvc` is the usual first failure.

That one element registers:

Registered	So that this works
`RequestMappingHandlerMapping` (order 0)	`@RequestMapping` is read
`RequestMappingHandlerAdapter`	annotated methods are invoked
the full `HttpMessageConverter` list, built from the classpath	`@RequestBody` / `@ResponseBody`, JSON via Jackson
`FormattingConversionService`	`@DateTimeFormat`, `@NumberFormat`, custom `Formatter`s
a JSR-303 `Validator`, if a provider is on the classpath	`@Valid` (Section 6)
`ExceptionHandlerExceptionResolver` + `ResponseStatusExceptionResolver` + `DefaultHandlerExceptionResolver`	`@ExceptionHandler`, `@ControllerAdvice`, `@ResponseStatus` (Section 7)

Level 2 — `@EnableWebMvc`

```java
@Configuration
@EnableWebMvc
@ComponentScan("com.example")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new RequestTimingInterceptor())
                .addPathPatterns("/contacts/**");
    }

    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver r = new InternalResourceViewResolver();
        r.setPrefix("/WEB-INF/views/");
        r.setSuffix(".jsp");
        return r;
    }

}
```

`@EnableWebMvc` is the *same thing* as `<mvc:annotation-driven/>`, expressed in Java. Same beans, same defaults, same trade-offs.

Mechanically it is one line:

```java
@Import(DelegatingWebMvcConfiguration.class)
public @interface EnableWebMvc { }
```

`DelegatingWebMvcConfiguration` extends `WebMvcConfigurationSupport` — which holds the actual `@Bean` methods — and collects every `WebMvcConfigurer` bean in the context, giving each one a callback into the setup. So:

- **`WebMvcConfigurer` is the Java counterpart of the `<mvc:...>` child elements.**
- **A `WebMvcConfigurer` does nothing on its own.** Something must import that configuration for it to be found.

---

1 · Request Mapping

`@RequestMapping` — The One Annotation Everything Builds On

```
@Controller
@RequestMapping("/contacts")            // class-level prefix, optional
public class ContactController {

    @RequestMapping(value = "/{id}", method = RequestMethod.GET)
    public String show(@PathVariable Long id, Model model) { ... }
}
```

`RequestMappingHandlerMapping` reads this at both levels and concatenates the paths. Everything else in this section is **sugar over this one annotation** — every shortcut you're about to see expands to it.

The Shortcuts

```
@GetMapping("/contacts")            // = @RequestMapping(method = GET)
@PostMapping("/contacts")           // = @RequestMapping(method = POST)
@PutMapping("/contacts/{id}")       // = @RequestMapping(method = PUT)
@DeleteMapping("/contacts/{id}")    // = @RequestMapping(method = DELETE)
@PatchMapping("/contacts/{id}")     // = @RequestMapping(method = PATCH)
```

They are literally `@RequestMapping` meta-annotated with one HTTP method fixed. Look at the source of `@GetMapping` and that's the whole implementation — a composed annotation, not new machinery:

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@RequestMapping(method = RequestMethod.GET)        // <- the entire
implementation
public @interface GetMapping {
}
```

So these two methods produce the **same** `RequestMappingInfo` **object** at runtime — same bean, same mapping, no difference whatsoever after startup:

```
@RequestMapping(value = "/search", method = RequestMethod.GET)   // long form
public String search(...) { ... }
```

```
@GetMapping("/search")                                          // shortcut
public String search(...) { ... }
```

```
@GetMapping(value = "/search", params = "mode=advanced", produces =
"application/json")
// identical to
@RequestMapping(value = "/search", method = RequestMethod.GET,
                params = "mode=advanced", produces = "application/json")
```

```
@RequestMapping(value = "/contacts", method = {RequestMethod.GET,
RequestMethod.HEAD})
```

And the bare form — no `method` at all — matches **every** HTTP verb. That is rarely what you want on a method, but it is exactly what you want at class level:

```
@RequestMapping("/contacts")        // a prefix, not a mapping
public class ContactController { }
```

Since 6.1, class-level path prefixes can also compose:

```
@RequestMapping("/contacts")
public class ContactController {
    @GetMapping                      // inherits /contacts
    @GetMapping("/{id}")             // -> /contacts/{id}
}
```

Query Parameters Are Not Part Of The Path

This trips almost everyone once. Given:

```
@GetMapping("/search")
public String search(...) { ... }
```

all of these match, identically:

```
/search
/search?q=spring
/search?q=spring&limit=3&tag=mvc&tag=jsp
```

The query string is invisible to path matching. You never write `@GetMapping("/search?q=...")` — that is not a thing, and `PathPatternParser` would treat the `?` as its single-character wildcard.

Two *different* mechanisms read the query string, and knowing which is which saves hours:

Mechanism	When it runs	What it is
`params = "..."` on the mapping	**during matching** — part of `RequestMappingInfo`	a condition. Wrong query string, and this method is never considered
`@RequestParam` on a parameter	**after** a handler was chosen	binding, done by `RequestParamMethodArgumentResolver`

The failure modes tell them apart instantly:

```
@GetMapping(value = "/action", params = "do=save")
public String save(@RequestParam String note) { ... }
```

- `/action?do=nothing` → **404**. No mapping matched; the handler never existed as far as the dispatcher is concerned.
- `/action?do=save` with no `note` → **400**. A handler *was* chosen; binding then failed.

404 means you got the `params` condition wrong. 400 means you got `@RequestParam` wrong.

---

2 · Getting Data In

Every Way A Parameter Can Be Filled

```
@PostMapping("/contacts/{id}")
public String update(
        @PathVariable Long id,                          // from the URL template
        @RequestParam String name,                       // from ?name=... or
form field
        @RequestHeader("User-Agent") String ua,          // from a request
header
        @CookieValue(value = "theme", required = false) String theme,
```

```
        @RequestBody Contact body)                    // the whole body,
convert
        { ... }
```

`@PathVariable` — The URL Template Itself

```
@GetMapping("/contacts/{id}")
public String show(@PathVariable Long id) { ... }          // name inferred,
needs -parameters
@GetMapping("/contacts/{id}")
public String show(@PathVariable("id") Long contactId) { ... }   // explicit,
always safe
```

Type conversion happens automatically through Spring's `ConversionService` — `String` → `Long` here needs nothing extra. An unparsable value throws `MethodArgumentTypeMismatchException`, which by default becomes a **400**, not a 500.

A missing path variable is impossible by construction: if the URL didn't match the pattern, the mapping wouldn't have matched at all.

`@RequestBody` and `@ResponseBody`

```
@PostMapping(value = "/contacts", consumes = "application/json")
@ResponseBody
public Contact create(@RequestBody Contact contact) { ... }
```

`@RestController` is `@Controller` + `@ResponseBody` applied to every method in the class — nothing more.

`@RequestHeader`, `@CookieValue`, `@RequestAttribute`

```
@GetMapping("/contacts")
public String list(@RequestHeader(value = "Accept-Language", required = false)
String lang,
                   @CookieValue(value = "sessionTheme", defaultValue =
"light") String theme,
```

```
                    @RequestAttribute(required = false) String probeMapping) {
... }
```

`@RequestAttribute` is the one worth pausing on: it reads a **request attribute**, not a parameter — the same mechanism a `HandlerInterceptor` uses to pass data to a handler (see Section 8).

`@ModelAttribute` As A Parameter — The Command Object

```
@PostMapping("/contacts")
public String create(@ModelAttribute Contact contact) { ... }
// or, equivalently, with nothing written at all:
@PostMapping("/contacts")
public String create(Contact contact) { ... }
```

If a parameter is **not** annotated and is **not** a simple type or a known framework type, Spring treats it as an implicit `@ModelAttribute`. This is the **command object** pattern, and it deserves its own section — next.

---

3 · Command Objects and Data Binding

What "Data Binding" Actually Means

A form posts flat strings:

```
name=Ada+Lovelace&email=ada%40example.com&birthDate=1815-12-10
```

A command object is a plain Java class:

```
public class Contact {
    private String name;
    private String email;
    private LocalDate birthDate;
    // getters and setters
}
```

```
@PostMapping("/contacts")
public String create(@Valid @ModelAttribute Contact contact, BindingResult
result) {}
```

Data binding is the mechanical process that turns the first into the second: match each parameter name to a JavaBean property, convert the string, call the setter. This is `WebDataBinder`'s entire job, and it runs automatically whenever a parameter is (implicitly or explicitly) `@ModelAttribute`.

`@InitBinder` — Per-Controller Binding Rules

```
@Controller
public class ContactController {

    @InitBinder
    public void initBinder(WebDataBinder binder) {
        SimpleDateFormat fmt = new SimpleDateFormat("dd/MM/yyyy");
        fmt.setLenient(false);
        binder.registerCustomEditor(Date.class, new CustomDateEditor(fmt,
true));

        binder.setDisallowedFields("id");     // security: never bind this
from the form
    }
}
```

Runs **before every handler method in this controller**, on every `WebDataBinder` it builds. Two common jobs: register a custom format, and use `setDisallowedFields` / `setAllowedFields` to close off properties a form should never be able to set.

The Command Object Isn't Only For Input

The exact same object you bound the form into is what you hand back to the view on a validation failure:

```
@PostMapping("/contacts")
public String create(@Valid @ModelAttribute Contact contact, BindingResult
result) {
    if (result.hasErrors()) {
```

```
        return "contact-form";          // "contact" is already in the model,
pre-filled
    }
    ...
}
```

Because the name defaults to the **uncapitalised class name** (`Contact` → `"contact"`), the form JSP can bind straight back to it without you adding it to the model by hand. `@ModelAttribute("customName")` overrides that default.

---

4 · Getting Data Out

`Model` , `ModelMap` , `ModelAndView` — Recap and the Choice

You want	Use
add attributes, return the view name as a `String`	`Model` — the ordinary case
the same, with `Map` -style access	`ModelMap`
compute the view name conditionally, or need both in one object	`ModelAndView`

```
@GetMapping("/contacts")
public String list(Model model) {
    model.addAttribute("contacts", service.findAll());
    return "contact-list";
}
```

Spring injects `Model` as a parameter — you never construct it. It is request-scoped: gone the instant the response is sent.

The Form Tag And The Command Object

```
<form:form modelAttribute="contact" method="post"
action="${pageContext.request.contextPath}/contacts">
```

```
    <form:label path="name">Name</form:label>
    <form:input path="name"/>
    <form:errors path="name" cssClass="error"/>

    <form:label path="email">Email</form:label>
    <form:input path="email" type="email"/>
    <form:errors path="email" cssClass="error"/>

    <input type="submit" value="Save"/>
</form:form>
```

`modelAttribute="contact"` must match the name the command object was bound under. Every inner tag's `path` is a **property path on that object** — `path="name"` means `contact.getName()` / `contact.setName()`, and it works through nested objects too: `path="address.city"`.

The Tag Vocabulary

Tag	Renders	Notable attribute
`<form:input>`	`<input type="text">`	`type="email"`, `type="date"` etc. work in HTML5 mode
`<form:password>`	`<input type="password">`	never re-populates the value, by design
`<form:textarea>`	`<textarea>`	
`<form:checkbox>`	a single boolean checkbox	binds to a `boolean` property
`<form:checkboxes>`	many checkboxes from a collection	`items="${statuses}"`
`<form:select>` + `<form:options>`	a `<select>`	`items`, `itemLabel`, `itemValue`
`<form:radiobutton>` / `<form:radiobuttons>`	radio inputs	
`<form:hidden>`	`<input type="hidden">`	commonly used for the `id` on an edit form
`<form:errors>`	validation messages	`path="*"` for all errors, or a specific property

---

Pre-Filling On Edit — Automatic, Not Manual

```
@GetMapping("/contacts/{id}/edit")
public String editForm(@PathVariable Long id, Model model) {
    model.addAttribute("contact", service.findById(id));   // same attribute
name
    return "contact-form";                                  // same JSP as
"create"
}
```

Because the JSP reads every field through `path="..."`, the **same form template** serves both create and edit — `<form:input path="name"/>` renders empty for a new `Contact`, pre-filled for an existing one. Nothing in the JSP changes between the two cases.

`<form:errors>` In Practice

```
<form:errors path="*" cssClass="error-summary"/>          <!-- every error,
one block -->
<form:errors path="email" cssClass="error"/>               <!-- only this
field's errors -->
```

`path="*"` is the common top-of-form summary; a `path` per field puts the message right where the user needs it. Both read from the same `BindingResult` — see Section 6 for where those messages come from.

---

5 · Validation

`@Valid` + `BindingResult` — The Contract

```
@PostMapping("/contacts")
public String create(@Valid @ModelAttribute Contact contact,
                     BindingResult result,
                     Model model) {
    if (result.hasErrors()) {
        return "contact-form";
    }
```

```
        service.save(contact);
        return "redirect:/contacts";
    }
```

`BindingResult` **must be the parameter immediately after the validated object.** If anything else comes between them, Spring throws instead of populating it — this is the single most common bug beginners hit with this feature.

`@Valid` triggers **JSR 380 / Jakarta Bean Validation**, run by `LocalValidatorFactoryBean` (auto-registered when a validation provider is on the classpath). No `@Valid` means validation never runs at all, silently.

Constraint Annotations — On The Command Object Itself

```
import jakarta.validation.constraints.*;

public class Contact {

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Not a valid email address")
    @NotBlank
    private String email;

    @Past(message = "Birth date must be in the past")
    private LocalDate birthDate;

    @Min(value = 0) @Max(value = 150)
    private int age;
}
```

Needs `jakarta.validation:jakarta.validation-api` plus an implementation — `org.hibernate.validator:hibernate-validator` — on the classpath. No `spring-boot-starter-validation` here since this project doesn't use Boot; the two jars are added directly.

Where The Errors Land

```
<form:errors path="name"/>    <!-- reads FieldError objects for the "name"
property -->
```

```
if (result.hasErrors()) {
    for (FieldError err : result.getFieldErrors()) {
        System.out.println(err.getField() + ": " + err.getDefaultMessage());
    }
}
```

Every failed constraint becomes a `FieldError` inside `BindingResult`, keyed by property name — which is exactly what lets `<form:errors path="email">` find only the errors for that one field.

Beyond One Field — Cross-Field Validation

Bean Validation annotations validate **one property**. Cross-field rules ("confirm password must match password") need a class-level constraint or Spring's own `Validator`:

```
@Component
public class ContactValidator implements
org.springframework.validation.Validator {

    @Override
    public boolean supports(Class<?> clazz) { return
Contact.class.equals(clazz); }

    @Override
    public void validate(Object target, Errors errors) {
        Contact c = (Contact) target;
        if (c.getBirthDate() != null &&
c.getBirthDate().isAfter(LocalDate.now().minusYears(13))) {
            errors.rejectValue("birthDate", "contact.tooYoung", "Must be at
least 13 years old");
        }
    }
}
```

```
@InitBinder
public void initBinder(WebDataBinder binder) {
```

```
    binder.addValidators(contactValidator);     // now @Valid runs this too
}
```

`errors.rejectValue(field, code, message)` attaches to a specific field, the same way a `@NotBlank` failure would — the JSP doesn't know or care which mechanism produced the error.

---

6 · Exception Handling

`@ExceptionHandler` — Scoped To One Controller

```
@Controller
public class ContactController {

    @GetMapping("/contacts/{id}")
    public String show(@PathVariable Long id, Model model) {
        model.addAttribute("contact", service.findById(id)
                .orElseThrow(() -> new ContactNotFoundException(id)));
        return "contact-detail";
    }

    @ExceptionHandler(ContactNotFoundException.class)
    public String handleNotFound(ContactNotFoundException ex, Model model) {
        model.addAttribute("message", ex.getMessage());
        return "error/not-found";
    }
}
```

An exception thrown by **any handler method in this class** is caught here instead of propagating. It is resolved by `ExceptionHandlerExceptionResolver`.

`@ControllerAdvice` — The Same Thing, Application-Wide

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ContactNotFoundException.class)
```

```
    public String handleNotFound(ContactNotFoundException ex, Model model) {
        model.addAttribute("message", ex.getMessage());
        return "error/not-found";
    }

    @ExceptionHandler(Exception.class)              // the catch-all, keep it
LAST / least specific
    public String handleUnexpected(Exception ex, Model model) {
        model.addAttribute("message", "Something went wrong.");
        return "error/general";
    }
}
```

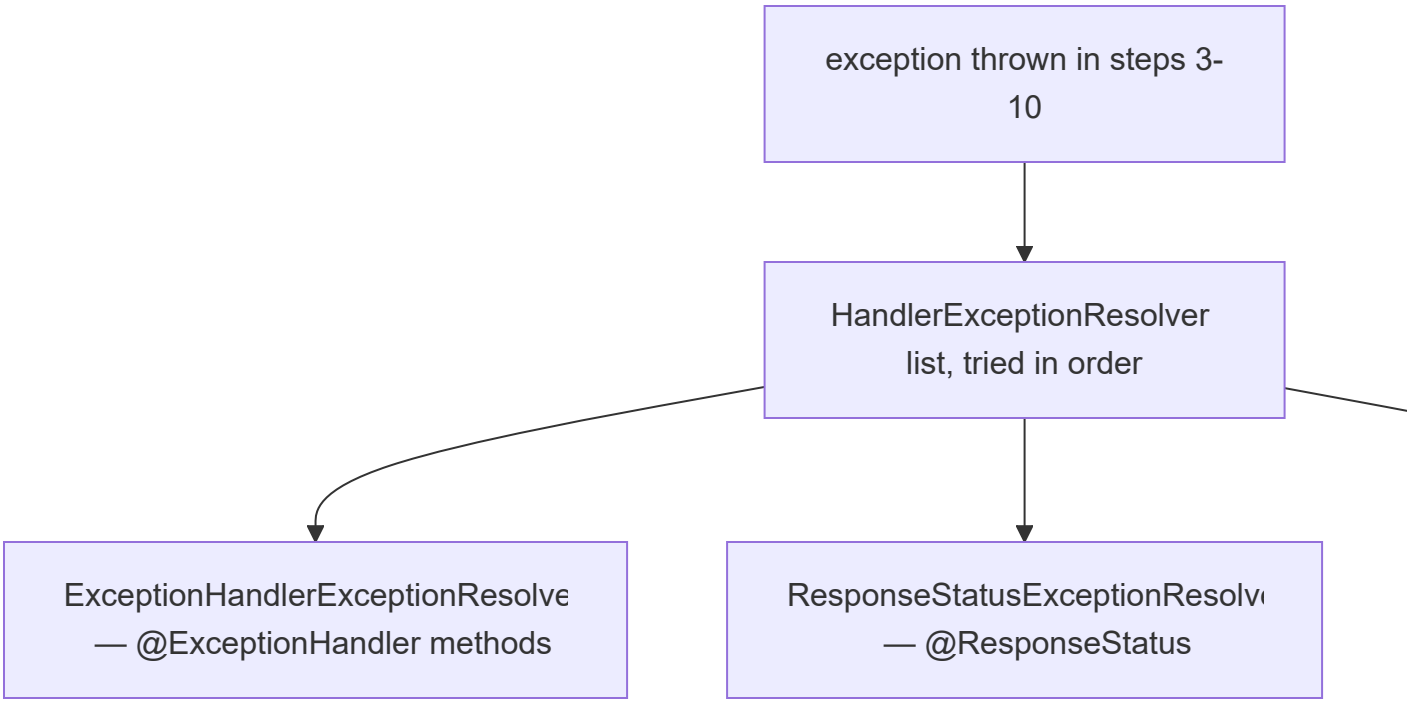
A `@ControllerAdvice` bean's `@ExceptionHandler` methods apply to **every** `@Controller` in the application, unless narrowed with `@ControllerAdvice(basePackages = "...")` or `assignableTypes = ...`. Spring picks the **most specific matching exception type** first, so order the methods from specific to general — it does not depend on declaration order.

`@ResponseStatus`

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ContactNotFoundException extends RuntimeException {
    public ContactNotFoundException(Long id) {
        super("No contact with id " + id);
    }
}
```

An even simpler option when you don't need a custom view: annotate the exception class itself, and an uncaught instance of it produces that status automatically, no handler method needed — resolved by `ResponseStatusExceptionResolver`, another entry in the same chain.

The Chain Behind Both Annotations



Same shape as every other seam in this framework: a `List`, sorted by order, first one to produce a `ModelAndView` wins. If none of them claims it, the exception reaches the container and you get its default error page.

7· `HandlerInterceptor` In Depth

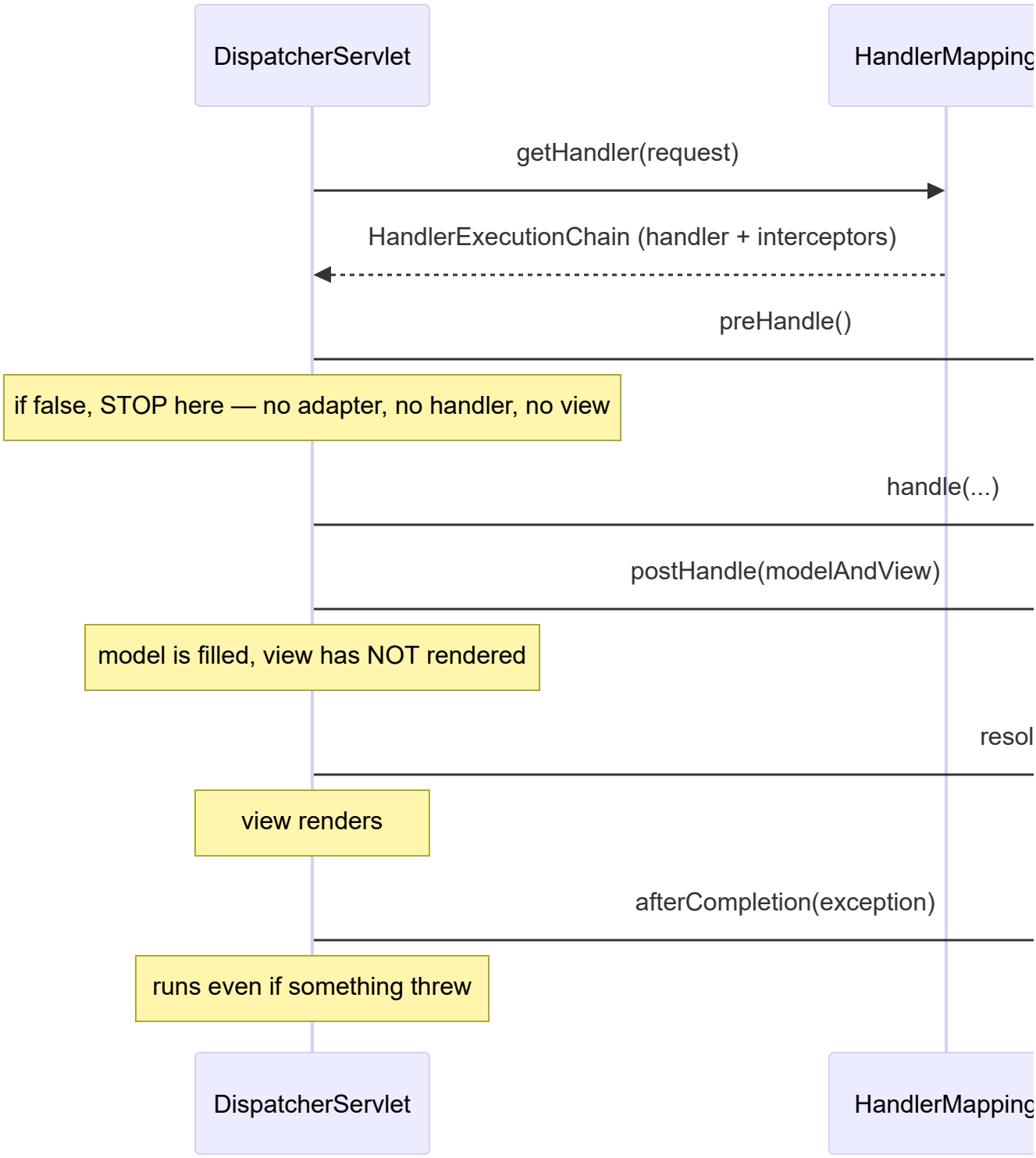
Where It Sits — Two Rings, Not One

“filter-vs-interceptor.svg” could not be found.

A **Filter** wraps the entire servlet call — it runs whether or not Spring, or `DispatcherServlet`, is even involved. A `HandlerInterceptor` wraps only the part *inside* `DispatcherServlet` between "handler chosen" and "view rendered". It is strictly the inner ring.

Exactly Where In The Lifecycle

Back to the Session 1 diagram, steps 4, 8 and 11:



Interceptors are **delivered by the mapping**, bundled into the `HandlerExecutionChain` — which is exactly why you register them on a `HandlerMapping` bean, never on `DispatcherServlet` directly.

The Three Hooks, Precisely

```
public interface HandlerInterceptor {

    default boolean preHandle(HttpServletRequest req, HttpServletResponse res,
Object handler) {
        return true;
    }

    default void postHandle(HttpServletRequest req, HttpServletResponse res,
                            Object handler, @Nullable ModelAndView
modelAndView) {
    }

    default void afterCompletion(HttpServletRequest req, HttpServletResponse
res,
                                 Object handler, @Nullable Exception ex) {
    }
}
```

Hook	Runs	`handler`	Can still change the view?	Runs on exception?
`preHandle`	before the adapter is even chosen	yes	N/A — nothing ran yet	—
`postHandle`	handler done, before rendering	yes	**yes** — add model attributes, even swap the view name	only if nothing threw
`afterCompletion`	after rendering	yes	no — response is committed	**yes, always**

Returning `false` from `preHandle` is a full stop: no adapter runs, no view renders. **You must write the response yourself** before returning `false`, or the browser gets nothing.

A Minimal, Honest Interceptor

```
public class RequestTimingInterceptor implements HandlerInterceptor {

    private static final String START = "timing.start";

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res,
```

```
Object handler) {
        req.setAttribute(START, System.nanoTime());
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse
res,
                                Object handler, Exception ex) {
        long startedAt = (long) req.getAttribute(START);
        long ms = (System.nanoTime() - startedAt) / 1_000_000;
        System.out.println(req.getRequestURI() + " took " + ms + " ms"
                + (ex == null ? "" : " (threw " +
ex.getClass().getSimpleName() + ")"));
    }
}
```

Notice `handler` is typed `Object`, same as everywhere else — to branch on annotations you must check `instanceof HandlerMethod` first, which only holds for `@Controller` handlers.

Registering One — Two Ways

Explicit bean on the mapping (the style Session 1's project used):

```
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="interceptors">
        <list><ref bean="requestTimingInterceptor"/></list>
    </property>
</bean>
```

`WebMvcConfigurer`, the modern way — registers on **every** mapping at once, with path filtering:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new RequestTimingInterceptor())
                .addPathPatterns("/contacts/**")
                .excludePathPatterns("/contacts/health");
    }
}
```

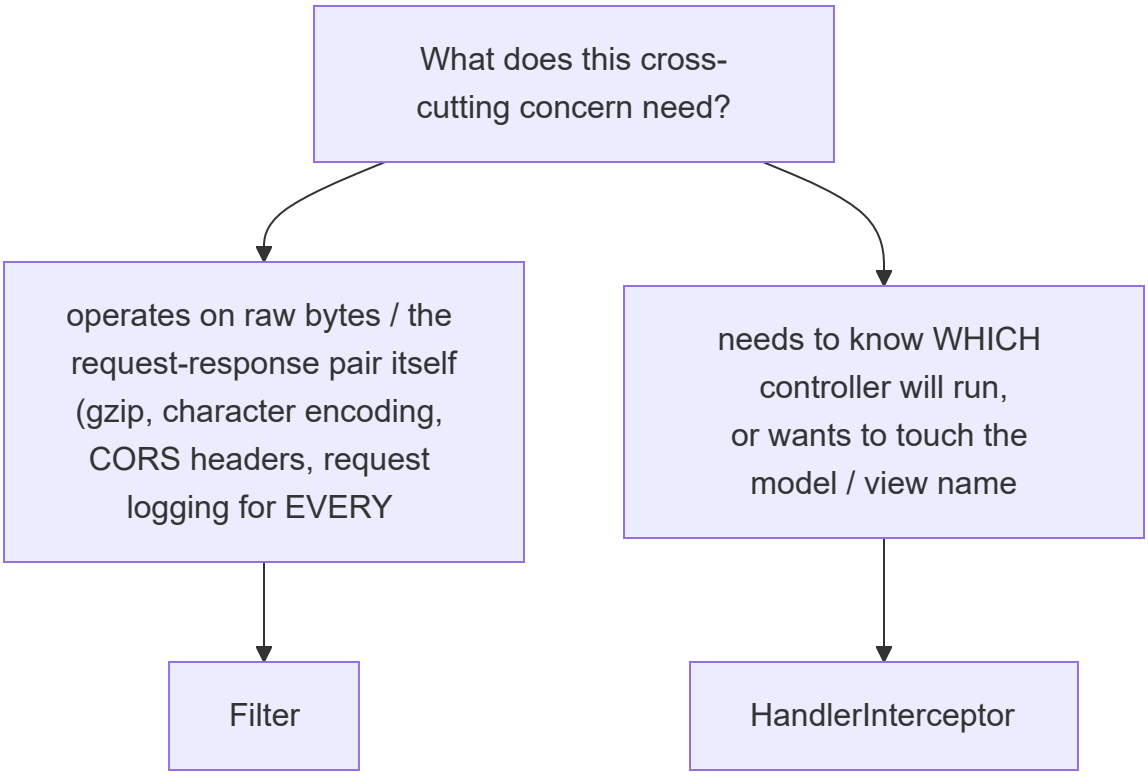
`addPathPatterns` / `excludePathPatterns` are interceptor-level — a Filter needs its own `url-pattern` in `web.xml` or a separate `FilterRegistrationBean` to do the same thing.

Why Does This Exist When We Already Have Filters?

Both intercept a request before "the real work" runs. The overlap is real — so is the reason they're separate.

Question	**Filter**	`HandlerInterceptor`
Defined by	Servlet API (`jakarta.servlet.Filter`)	Spring MVC
Knows about Spring beans?	only if you fetch the `ApplicationContext` yourself	**it is a Spring bean** — full DI
Sees which handler will run?	no	**yes** — the `Object handler` parameter
Sees the model / view name?	never	**yes**, in `postHandle`
Runs for JSPs, static files, other servlets?	**yes**	no — only requests `DispatcherServlet` dispatches
Can swap the request/response object?	**yes** — its `doFilter(req, res, chain)` signature allows wrapping	no — works with what it's given
Configured	`web.xml` or `@WebFilter` / `FilterRegistrationBean`	a `HandlerMapping` bean or `WebMvcConfigurer`

Same Table, As A Decision



Rule of thumb: if the concern is about the *transport* (encoding, compression, raw security headers), it's a Filter. If the concern is about *this specific request's handler or view* (per-handler auth checks, adding a model attribute everywhere, measuring only MVC-handled requests), it's an interceptor.

They Compose — This Isn't Either/Or

A real application layers both, and each does what it's naturally suited to:



Security's coarse "is there a session at all" check is commonly a Filter (it must run for every resource); a finer "does this user's role allow this specific handler" check, which wants to inspect the `HandlerMethod`, fits naturally as an interceptor.